

Python RegEx Comprehensive Guide

Master Regular Expressions with Python's 're' Module

Powered by codehive

1. Introduction to Regular Expressions

A RegEx, or **Regular Expression**, is a sequence of characters that forms a search pattern. RegEx can be used to check if a string contains the specified search pattern.

In Python, the built-in package `re` provides all the necessary tools to work with Regular Expressions.

```
import re
```

Basic Example

Search the string to see if it starts with "The" and ends with "Spain":

```
import re
txt = "The rain in Spain"
x = re.search("^The.*Spain$", txt)
if x:
    print("YES! We have a match!")
else:
    print("No match")
```

2. The Core Functions

The `re` module offers a set of functions that allows us to search a string for a match:

Function	Description
findall	Returns a list containing all matches
search	Returns a Match object if there is a match anywhere in the string
split	Returns a list where the string has been split at each match
sub	Replaces one or many matches with a string

The `findall()` Function

The `findall()` function returns a list containing all matches in the order they are found. If no matches are found, an empty list is returned.

```
import re
txt = "The rain in Spain"
x = re.findall("ai", txt)
print(x) # ['ai', 'ai']
```

3. Metacharacters

Metacharacters are characters with a special meaning in a search pattern.

Char	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence / escape	"\d"
.	Any character (except newline)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"planet\$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrences	"he.?o"
{}	Exactly the specified number	"he.{2}o"
	Either or	"falls stays"
()	Capture and group	(abc)

4. Special Sequences

A special sequence is a `\` followed by one of the characters below:

Char	Description	Example
<code>\A</code>	Match if at the start of string	<code>"\AThe"</code>
<code>\b</code>	Match at start or end of a word	<code>r"\bain"</code>
<code>\B</code>	Match if NOT at start/end of word	<code>r"\Bain"</code>
<code>\d</code>	Digits (0-9)	<code>"\d"</code>
<code>\D</code>	NON-digits	<code>"\D"</code>
<code>\s</code>	White space character	<code>"\s"</code>
<code>\S</code>	NON-white space	<code>"\S"</code>
<code>\w</code>	Word chars (a-Z, 0-9, and <code>_</code>)	<code>"\w"</code>
<code>\W</code>	NON-word characters	<code>"\W"</code>
<code>\Z</code>	Match if at the end of string	<code>"Spain\Z"</code>

5. Character Sets

A set is a set of characters inside square brackets `[]` with a special meaning.

Set	Description
<code>[arn]</code>	Returns match where a, r, or n are present
<code>[a-n]</code>	Lower case alpha between a and n
<code>[^arn]</code>	Any character EXCEPT a, r, and n
<code>[0123]</code>	Digits 0, 1, 2, or 3
<code>[0-9]</code>	Any digit between 0 and 9
<code>[0-5][0-9]</code>	Two-digit numbers from 00 and 59
<code>[a-zA-Z]</code>	Any alphabet char regardless of case
<code>[+]</code>	The character '+' (Special chars lose meaning)

6. Match Objects & Methods

A Match Object contains information about the search and the result. If no match is found, `None` is returned.

- **.span():** returns a tuple containing the start and end positions.
- **.string:** returns the string passed into the function.
- **.group():** returns the part of the string where there was a match.

```
import re
txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.span()) # (12, 17)
print(x.string) # The rain in Spain
print(x.group()) # Spain
```

7. Advanced Functions: Split & Sub

The split() Function

Splits the string at each match. You can use `maxsplit` to control occurrences.

```
# Split at the first occurrence only
x = re.split("\s", txt, 1)
```

The sub() Function

Replaces matches with text of your choice. Use `count` to limit replacements.

```
# Replace first 2 occurrences of space with '9'
x = re.sub("\s", "9", txt, 2)
```


8. RegEx Flags

Flags modify the behavior of the pattern matching.

Flag	Shorthand	Description
re.IGNORECASE	re.I	Case-insensitive matching
re.MULTILINE	re.M	^ and \$ match start/end of each line
re.DOTALL	re.S	Makes '.' match newline too
re.VERBOSE	re.X	Allows whitespace and comments in RegEx
re.ASCII	re.A	Perform ASCII-only matching

9. Practical Exercise

Knowledge Check

Consider the following code:

```
import re
txt = 'The rain in Spain'
x = re.findall('[a-c]', txt)
print(x)
```

What will be the printed result?

Answer: ['a', 'a']

Explanation: The pattern `[a-c]` looks for characters 'a', 'b', or 'c'. In 'The rain in Spain', 'a' occurs twice (in 'rain' and 'Spain'). There are no 'b's or 'c's.